Dinner Democracy

Your family's setup & launch guide

Your next step: make sure the new application code is saved to GitHub, then deploy it with the included Render configuration. Render will provide the secure address that Shani, James and Xander can open on their phones.

Start with the seven Render steps below. The technical alternative, backup advice and full validation checklist are included in this PDF, so you do not need to open a Markdown file.

Your repository: shanidelamor/dinner-democracy · Render dashboard

Get the family's phone link

This workspace is a development environment. Running a server here does **not** publish an internet-accessible family app. No hosting credentials were supplied and no public deployment has been made.

Recommended: one Render web service with a persistent disk

1. Commit/review the source in your GitHub repository and push it when you are ready.
2. In your Render account, create a Blueprint from the repository using `render.yaml`. It describes one paid Starter Docker web service and a 1 GB persistent disk at `/app/data`. Check Render's current pricing before deploying.
3. Render generates a private `SETUP_KEY` environment variable. Open the service's environment settings to obtain that key securely; do not paste it in chat or commit it.
4. Deploy. Render provides the HTTPS application address. Check `/api/health` returns `{"status":"ok","storage":"sqlite"}`.
5. Open the HTTPS app address, supply the setup key, and choose the three profile passwords. The setup endpoint refuses reinitialization once users exist.
6. Share the HTTPS address with James and Xander; send each person only their own password privately. Each uses the appropriate profile on their phone. No installation is required.

7. Run the physical-device checks in the validation section later in this PDF. Keep the service and its persistent disk; a throwaway filesystem will lose data on redeploy.

The Render Blueprint is provided but was not deployed or tested against a Render account in this environment. Use **one instance** with this SQLite architecture. Multiple independent replicas or serverless request handlers need a shared managed SQL database adaptation.

Alternative: any Docker host with HTTPS and a persistent volume

```
docker build -t dinner-democracy .
# Set SETUP_KEY privately in your shell or secret manager, then pass its name only.
docker run -d --name dinner-democracy --restart unless-stopped \
  -p 3001:3001 -e SETUP_KEY \
  -v dinner-democracy-data:/app/data dinner-democracy
```

Terminate HTTPS at the host/reverse proxy. The app trusts one proxy hop and uses Secure, HttpOnly, SameSite=Lax session cookies in production; plain HTTP production logins will not work. Preserve the original Host and Origin and forward requests to port 3001. The Docker image runs as the unprivileged `node` user; ensure the persistent data path is writable by UID 1000. The Docker image was built successfully in this workspace. See the validation section later in this PDF for the production-container smoke-check result.

Environment variables

Name	Purpose
`NODE_ENV=production`	Secure cookies, production headers, protected first setup
`PORT`	HTTP listen port; default 3001, all interfaces
`DATABASE_PATH`	Persistent SQLite path; Docker defaults to `/app/data/dinner-democracy.sqlite`
`SETUP_KEY`	Private first-setup authorization, required in production

For local development, optionally put these settings in an ignored `.env`, and start Node using `node --env-file=.env server/index.js`. No third-party API credential is required. Generated illustrative food images and fonts are local, so application use requires no third-party image service.

Backups and upgrades

Stop writes and use SQLite's backup API or the CLI `.backup` command for a consistent backup. Copying only a live `.sqlite` file while its WAL is active is unsafe. Keep encrypted backups outside the service and verify restoration on a separate database. Recipes, profile password hashes, session hashes, requests and voting history are all household data.

Before deploying an upgrade, back up the database. Current startup schema creation and recipe seeding are idempotent and do not reset data; future schema changes should add explicit numbered migrations. Do not delete the database to fix an upgrade problem. For a deployment transfer, copy the database consistently and update DATABASE_PATH; keep the hostname stable where practical, and sign in again if needed.

Boundaries of the first version

- This is a single trusted household, not a multi-household SaaS. Members have separate passwords and own-request restrictions; Shani has admin privileges.
- Shopping quantities describe ingredients to consume, not live Trader Joe's prices or guaranteed package availability. Adjust purchase amounts for the products on the shelf.
- No push/email notifications or external recipe-conversation import: request statuses appear in the app, and household state refreshes every 15 seconds and on window focus.
- The ingredient catalog has canonical products and cuts. New recipes can use that catalog; adding entirely new ingredient definitions currently requires a catalog code update.
- Steps in cooking mode are current-screen checkmarks, while recipes, votes, menus, requests, pantry and shopping checkboxes persist centrally.
- Cooking times, portions and reconstructed recipes need Shani's kitchen review. Raw poke requires supplier-confirmed safe handling; choose cooked fish when uncertain.

Cloud build trust configuration

This onboarding environment uses an outbound package proxy with an additional trusted CA. Its container builder needed the proxy hostname to be resolved and that public trust certificate passed through the optional `npm_ca` build secret. The certificate is mounted only while installing packages and is not copied into the final image. TLS verification and lockfile integrity checks remain enabled. Ordinary Docker hosts and Render can use their standard trusted network without supplying this optional certificate. Do not disable TLS verification to work around a corporate or cloud proxy.

For future Codex sessions

The reusable installation and startup instructions were saved in the cloud environment draft. In Codex, open **Edit environment**, review and save those changes, then publish the environment for future coding tasks.

Publishing the Codex environment preserves your development setup. Deploying the application to Render provides the web address your family will use.

Validation & first-phone checklist

The current source passes `npm test` (**19 passing tests/subtests; zero failures/skips**) and `npm run build` (TypeScript + Vite). A Playwright Chromium test passes the complete family workflow using **three independent browser contexts**, including two phone-size viewports. These are simulated devices, not physical phone testing.

Test accounts and votes live only in disposable test databases. The real workspace database is left at first-run setup, with no invented household passwords or votes.

Requested workflow	Evidence
Shani opens a voting round	Browser UI and API test
Xander votes separately	Independent mobile-sized browser context and own cookie session
James votes independently	Third browser context and own cookie session
Votes survive refresh/reopening	Browser reload; API test stops/reopens server and SQLite file, reuses sessions
Correct combined results	Browser checks individual YES counts; API verifies three YES votes for one recipe
Editable weekly plan	Browser generates 7 days and changes portions; API tests locks, regeneration and approval
Servings change ingredients	Exact domain assertions on chicken, rice, sauce and broccoli; browser recipe adjuster
Vegetable sides appear in groceries	Browser checks broccoli; domain tests assert exact scaled vegetable quantities

Requested workflow	Evidence
Quantities consolidate correctly	Compatible weight/volume arithmetic and distinct-cut assertions; linked leftover batch counted once
Xander asks for two boxes of soda	Multi-line API request and browser request form; parsed quantity preserved
Shani approves and adds two boxes	Browser explicitly confirms shopping conversion; API retries conversion to verify idempotency
James asks for cookies; Shani schedules	Separate browser sessions; baking ingredients appear only after plan approval
Shopping checks persist	Browser checks Soda and reloads; API verifies checkbox after server/database restart
Printable menu and groceries	Chromium prints fridge-menu, 14-day-menu and shopping PDFs; CSS checks preserve vegetable text
New weeks preserve history	Browser reselects prior round; API asserts old votes survive opening a new round

Additional checks cover unauthenticated access, member/admin restrictions, other members' request-edit denial, cross-origin mutation denial, invalid calendar dates, James/seafood conflicts, closed-round vote rejection, recipe edit history, new recipe creation/archival, missing-vegetable rejection, stale grocery-check reset after recipe changes, 14-day generation and linked leftover safety.

Artifacts are written under `test-results/family-three-independent-f-8fac1-the-core-household-workflow/` by the current browser run: `home-desktop.png`, `home-mobile.png`, `vote-mobile.png`, `fridge-menu.pdf`, `fridge-menu-14-days.pdf`, and `shopping-list.pdf`. This directory is ignored by Git and regenerated on each test run. Locally served food photos were verified to load; they do not require external image-host access.

The actual development server was started using `npm run dev`. The frontend, proxied status/health endpoints, SQLite health response and local salmon image returned HTTP 200. Frozen dependency reinstall (`npm ci`) and rebuild were exercised successfully. Reusable `install_script` and `start_skill` fields were saved to the cloud environment draft; saving them does not publish the environment or host the application publicly.

The 7-day menu with a baking task and the 14-day fridge menu each printed on **two landscape A4 pages** in Chromium. Main meal descriptions use 12-point print text. The grocery list groups items by store section and excludes checked/already-have items when printing.

The production Docker image also built successfully. `tests/docker-smoke.js` launched it with an isolated persistent volume and a randomly generated setup key, verified missing-key rejection, Secure/HttpOnly cookie flags, the built frontend and local image, and preserved a vote/session across container restart. The test then removed only its own synthetic container and volume. No HTTPS browser or external hosting deployment is implied by this check.

Still needs user/device validation

- Deploy the app to a single HTTPS service with a persistent disk using the Render steps earlier in this PDF; no public hosting deployment was performed.
- On real phones, sign in separately as Shani, James and Xander. Cast different votes and confirm results on Shani's device, after both refresh and closing/reopening the browser.
- Check touch swipes, installed-browser behavior, landscape/portrait layouts, a physical printer, and wake-lock support. Browser tests exercise voting buttons, responsive layouts and generated PDFs; they do not prove all touch hardware or printer behavior.
- Review recipe reconstructions, allergen/GF product labels, appropriate food handling, ingredient availability, package sizes and generous serving yields in your kitchen.
- Verify the chosen host's disk survives redeployment and test restoration from a consistent backup. A restart of the local API/database has been tested; no external host's persistence is claimed.
- GitHub Actions configuration is included but has not run on GitHub from this workspace.